

PAP 2.2 – Python-Auswertung

Wichtige Funktionen für das PAP.

Alle in diesem Dokument geschriebenen Zeilen Code sind von mir eigenständig geschrieben. Für ein sauberes und strukturiertes Dokument wurde eine eigene Pythonlibrary geschrieben, welche insbesondere ans PAP angepasst wurde. Dieses ist unter dem Namen "Papulator" auf meinem GitHub zu finden. Diese soll stetig ausgearbeitet werden. Der Stand dieses Protokolls ist der 29.04.2026.

Definition der Versuchsvariablen

Im Folgenden sind Versuchsvariablen definiert, die für einen besseren Workflow sorgen sollen. Diese werden zum exportieren und überschreiben von Dateien wichtig sein und ermöglichen es, diese Datei für jeden Versuch zu benutzen und lediglich die Variablen zu verändern. Es ist jedoch empfohlen eine Kopie der Forlage für jeden Versuch zu machen, damit dieses Dokument strukturiert bleibt.

Zudem sind wichtige Konstanten definiert, die immer wieder auftauchen.

```
In [100... versuchsnummer = "256"
versuchsname = "X-ray flourescence"
aufgabe = "0z"
plt_save_folder = f"../img/plots/"
```

Import aller genutzen Libraries

```
In [101... # Eigene PythonLibrary für das Pap
import Papulator as pap
from Papulator import Colors as c
from Papulator import const
from Papulator import Sympy_Symbols as sym

# Numpy für bessere Berechnungen
import numpy as np
from numpy import exp, sqrt, log, pi
from uncertainties import unumpy as unp

# Weiteres für bessere Rechnungen
import pylab as py

import math

# Berechnungen und Plotting
from scipy import odr
import scipy.optimize
from scipy.optimize import curve_fit
from scipy.stats import norm
from scipy.stats import chi2
from scipy.stats import poisson
from scipy.signal import find_peaks
from scipy.signal import argrelextrema, argrelemin, argrelemax
from scipy.special import factorial
from scipy.integrate import quad

import matplotlib.pyplot as plt
import matplotlib.mlab as mlab
import matplotlib.transforms as transforms

import astropy.units as u

# Zum Auslesen von Dateien und ähnlichem
import os
import os.path

import pandas as pd
import csv
import re

from io import StringIO

# Besseres Funktionen handling
import sympy as sp
from sympy import separatevars
```

Auswertung der Aufgaben

```
In [102... element = ['Fr', 'Mo', 'Ag', 'Cu', 'Ni', 'Ti', 'Zn', 'Zr']
charge_Z = np.array([26, 42, 47, 29, 28, 22, 30, 40])

K_alpha = np.array([6.40, 17.45, 21.80, 8.05, 7.47, 4.67, 8.65, 15.87])
err_K_alpha = np.array([16, 18,19,17,17,16,17,17]) * 1e-2
sqrt_K_alpha=np.sqrt(K_alpha)
err_sqrt_K_alpha =err_K_alpha/(2*np.sqrt(K_alpha))

K_beta = np.array([7.03, 19.54, 24.53, 8.89, 8.26, np.nan, 9.59, 17.96])
err_K_beta = np.array([18, 23,14,20,18,np.nan,16,20]) * 1e-2

sqrt_K_beta=np.sqrt(K_beta)
err_sqrt_K_beta =err_K_beta/(2*np.sqrt(K_beta))
```

```
In [103... n1 = 1
n2 = 2

def fit_func(x,sqrt_Er, sig12):
    return sqrt_Er*(x-sig12)*np.sqrt(1/n1**2-1/n2**2)
```

```
In [104... plt.figure(figsize=(12,7))

plt.errorbar(
    charge_Z,
    sqrt_K_alpha,
    err_sqrt_K_alpha,
    ls= ' ',
    capsize=3.5,
    label='Messurment Data'
)

# Regression
popt, pcov=curve_fit(fit_func, charge_Z, sqrt_K_alpha, sigma=err_sqrt_K_alpha)

E_R_rooted = pop[0]*np.sqrt(1000) # eV
err_E_R_rooted = np.sqrt(pcov[0,0] * 1000)

print(f'E_R rooted= ({pap.round_sig_digs(err_E_R_rooted, E_R_rooted)[2]}) eV')

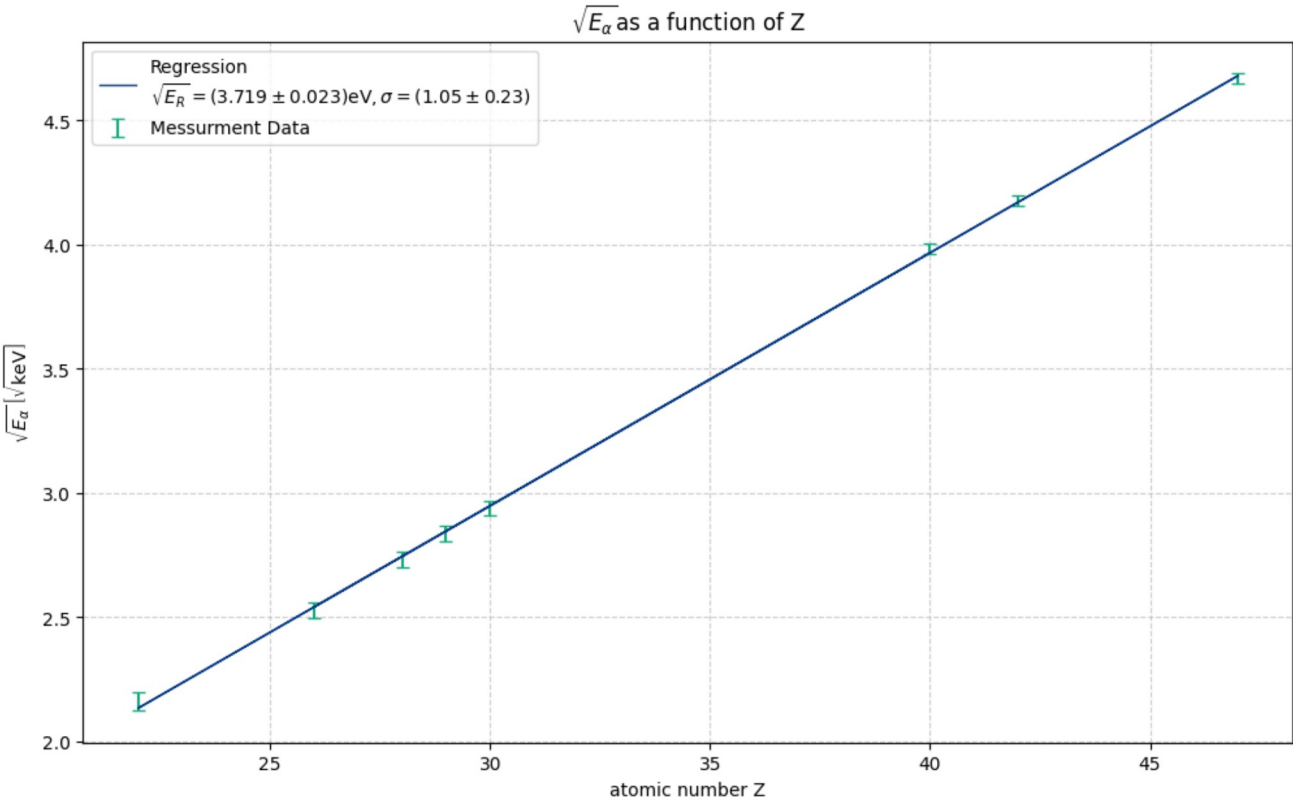
sigma12 = pop[1]
err_sigma12 = np.sqrt(pcov[1,1])

print(f'sigma = ({pap.round_sig_digs(err_sigma12, sigma12)[2]})')

plt.plot(charge_Z,
    fit_func(charge_Z,*popt),
    label = f"Regression\n$\sqrt{E_R}$ = ({pap.round_sig_digs(err_E_R_rooted, E_R_rooted)[2]}) \mathrm{eV}, \sigma = ({pap.round_sig_digs(err_
    lw=1
)

pap.plot_me(r'$\sqrt{E_\alpha}$'+as a function of Z', 'atomic number Z', r'$\sqrt{E_\alpha}$ \, \left[\mathrm{\sqrt{keV}}\right]$', f'{plt_save_folder}Al

E_R rooted= (3.719 \pm 0.023) eV
sigma = (1.05 \pm 0.23)
```



```
Out[104... <Axes: title={'center': '$\sqrt{E_\alpha}$as a function of Z'}, xlabel='atomic number Z', ylabel='$\sqrt{E_\alpha}$ \, \left[\mathrm{\sqrt{keV}}\right]'
```

```
In [105... E_R = E_R_rooted**2
err_E_R = 2 * err_E_R_rooted * E_R_rooted

print(f'E_R = ({pap.round_sig_digs(err_E_R, E_R)[2]}) eV')
```

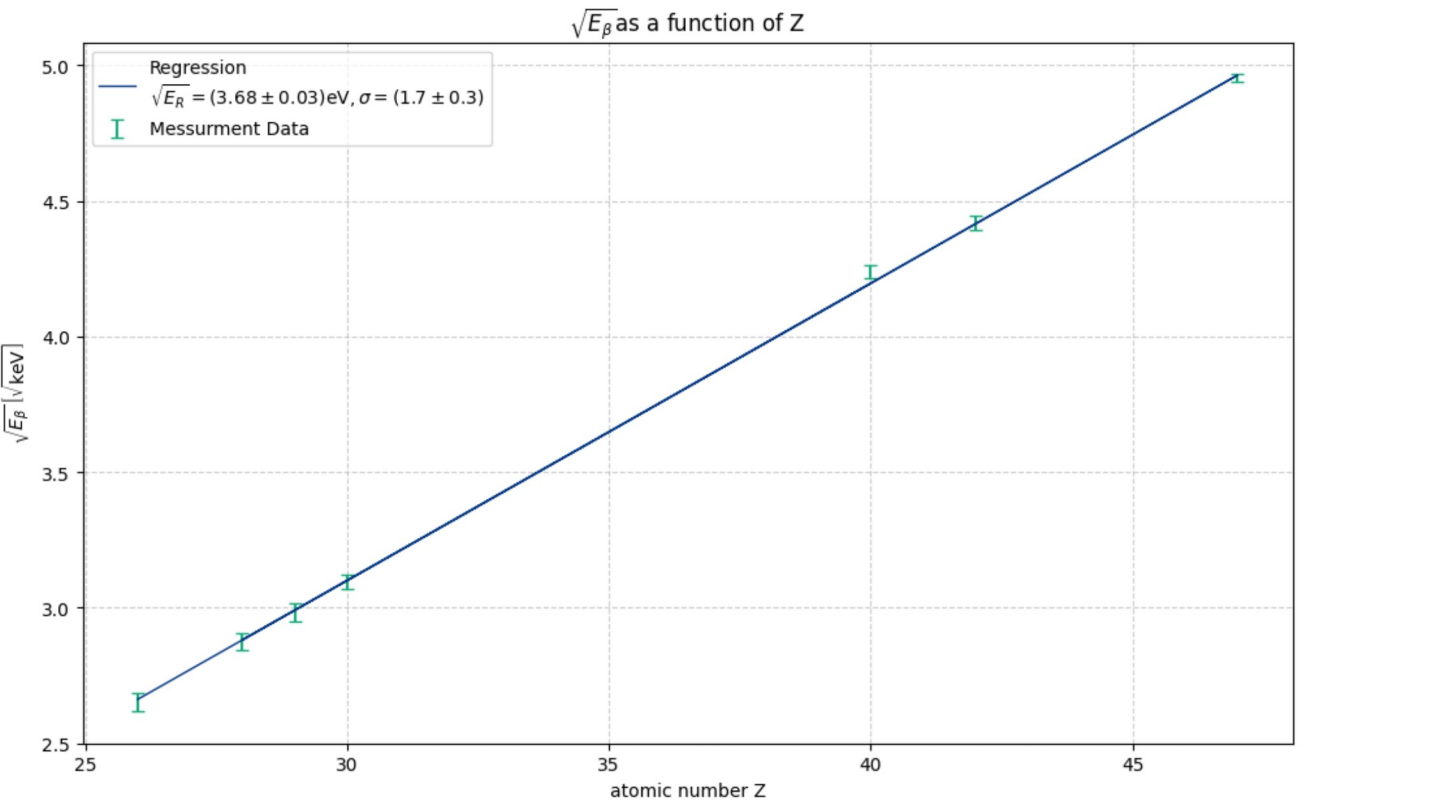
```
print(f'standard dviation form lit: {pap.std_abw(E_R, 13.606, err_E_R)} \\\sigma')
```

```
E_R = (13.83 \pm 0.17) eV  
standart dviation form lit: 1.3 \sigma
```

K Beta

```
In [106...  
element = ['Fr', 'Mo', 'Ag', 'Cu', 'Ni', 'Zn', 'Zr']  
charge_Z = np.array([26, 42, 47, 29, 28, 30, 40])  
  
K_beta = np.array([7.03, 19.54, 24.53, 8.89, 8.26, 9.59, 17.96])  
err_K_beta = np.array([18, 23,14,20,18,16,20]) * 1e-2  
  
sqrt_K_beta=np.sqrt(K_beta)  
err_sqrt_K_beta =err_K_beta/(2*np.sqrt(K_beta))  
  
# Value for Ti was removed
```

```
In [107... # For the regression  
n1 = 1  
n2 = 3  
  
plt.figure(figsize=(12,7))  
  
plt.errorbar(  
    charge_Z,  
    sqrt_K_beta,  
    err_sqrt_K_beta,  
    ls= ' ',  
    capsize=3.5,  
    label='Messurment Data'  
    )  
  
# Regression  
popt, pcov=curve_fit(fit_func, charge_Z, sqrt_K_beta, sigma=err_sqrt_K_beta)  
  
E_R_rooted = pop[0]*np.sqrt(1000) # eV  
err_E_R_rooted = np.sqrt(pcov[0,0] * 1000)  
  
print(f'E_R rooted= ({pap.round_sig_digs(err_E_R_rooted, E_R_rooted)[2]}) eV')  
  
sigma12 = pop[1]  
err_sigma12 = np.sqrt(pcov[1,1])  
  
print(f'sigma = ({pap.round_sig_digs(err_sigma12, sigma12)[2]})')  
  
plt.plot(charge_Z,  
    fit_func(charge_Z,*pop),  
    label = f"Regression\n$\sqrt{{{E_R}}}$ = ({pap.round_sig_digs(err_E_R_rooted, E_R_rooted)[2]}) \\\mathrm{{{eV}}}, \\\sigma = ({pap.round_sig_digs(err_",  
    lw=1  
    )  
  
pap.plot_me(r'$\sqrt{{{E_\beta}}}$'+as a function of Z', 'atomic number Z', r'$\sqrt{{{E_\beta}}}$ \, \left[\\\mathrm{\sqrt{keV}}\right]$', f'{plt_save_folder}Beta',  
    )  
  
E_R rooted= (3.68 \pm 0.03) eV  
sigma = (1.7 \pm 0.3)
```



```
Out[107... <Axes: title={'center': '$\sqrt{{{E_\beta}}}$as a function of Z'}, xlabel='atomic number Z', ylabel='$\sqrt{{{E_\beta}}}$ \, \left[\\\mathrm{\sqrt{keV}}\right]$',
```

In [108...

```
E_R = E_R_rooted**2
err_E_R = 2 * err_E_R_rooted * E_R_rooted

print(f'E_R = ({pap.round_sig_digs(err_E_R, E_R)[2]}) eV')

print(f'standard dviation form lit: {pap.std_abw(E_R, 13.606, err_E_R)} \\sigma')
```

```
E_R = (13.51 \pm 0.24) eV
standard dviation form lit: 0.39 \sigma
```